Why Java? [What is Java?]

Java is a programming language and a platform. It is a High level, robust, object-oriented and Secure programming language.

→ Java has a runtime environment (JRE) and API, it is called a platform.

History of Java

1. Java project initiated in 1991 by James Gosling, Mike Sheridan and Patrick Naughton known as the Green Team at Sun Micro System.

2. Initially designed for small embedded systems like set-up boxes

3. Originally called "Green talk" with file extension .gt.

4. Renamed to Oak (Oak symbolizes strength) (National tree of several Countries).

5. Renamed to Java in 1995 due to trademark issues with Oak technologies.

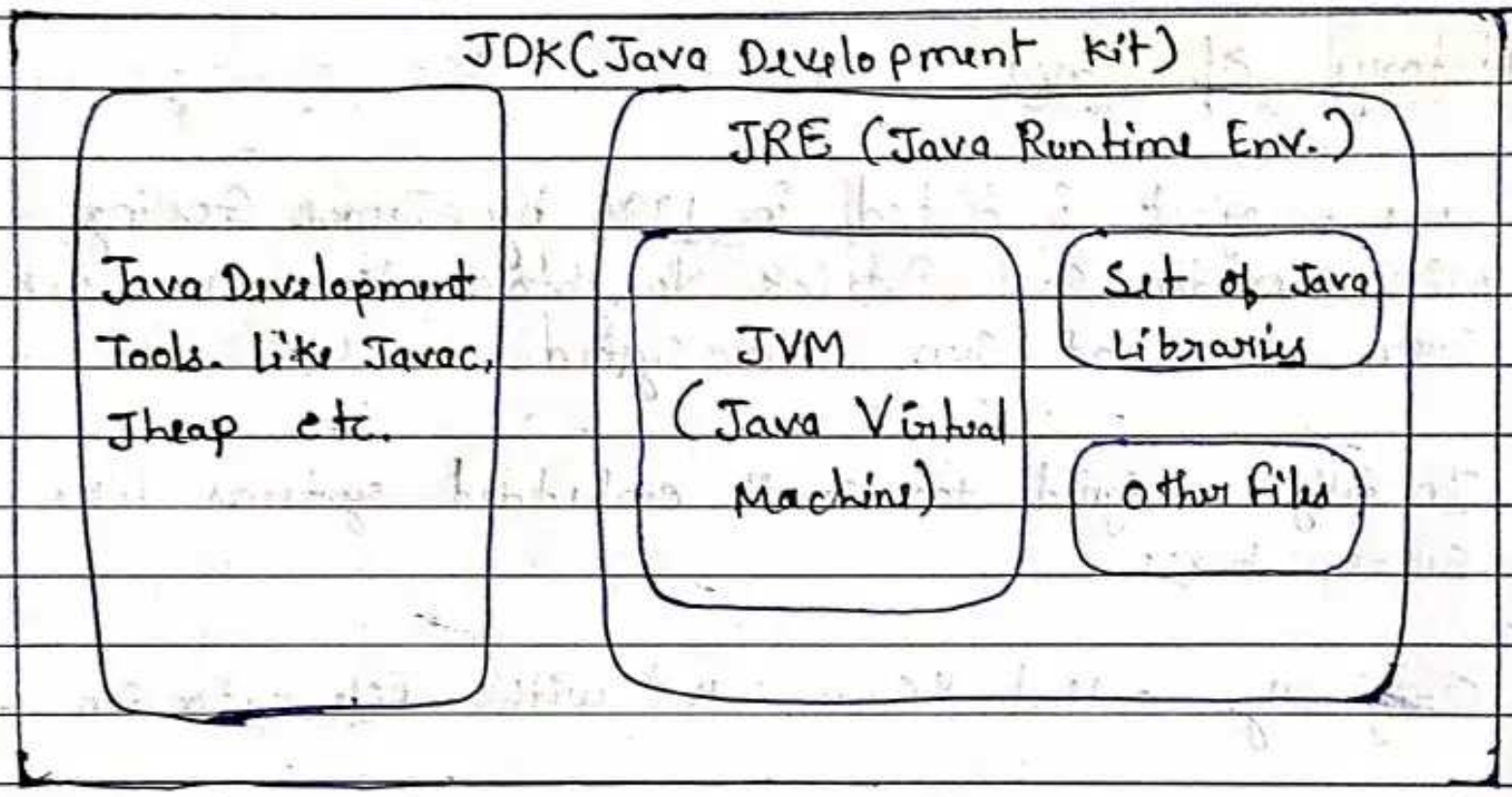
6. Named after Java Island in Indonesia, where the first coffee was produced.

7. Recognized by Time magazine as one of the Ten Best Products of 1995.

Date

→ Features of Java
- → Simple
- → object Oriented
- → Robust
- → Platform Independent
- → Secured
- → Multithreaded.

→ JDK Vs JVM Vs JRE



① JDK
- → It's a software development kit required to developed Java applications.
- → Includes the JRE, an interpreter, a compiler and other tools needed for Java development.
- → JDK is super set of JRE.

Date

② JRE

→ It's a part of the JDK but can be downloaded separately.
→ Provides the libraries, the JVM and other components to run application.
→ Does not have tools and utilities for developers like Compilers

③ JVM

→ It is an abstract machine. It is a specification that provides runtime environment in which Java bytecode can be executed.
→ It is platform dependent

The JVM performs following operation :

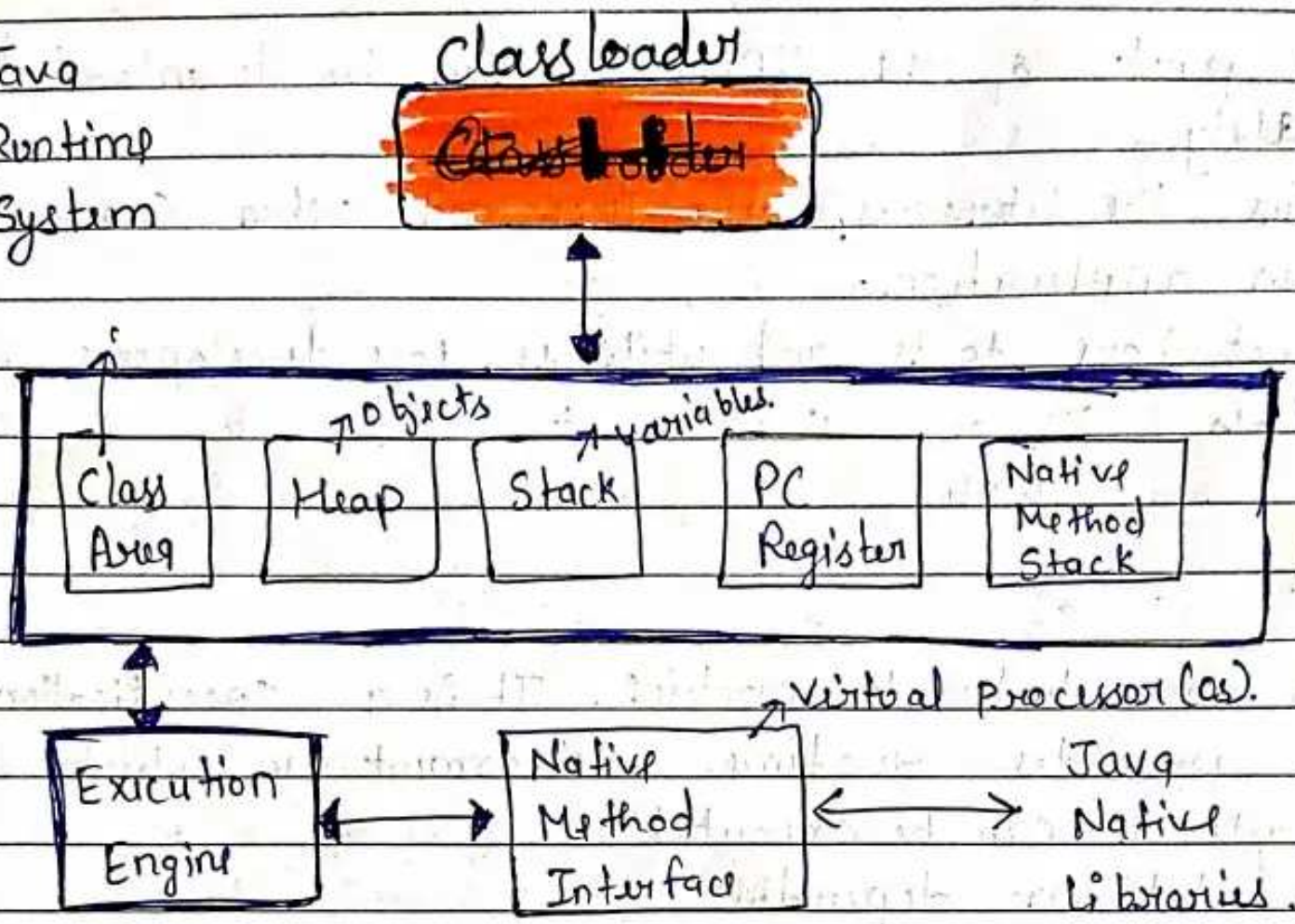
- Loads code
- Verifies code
- Executes code
- Provides runtime environment.

JVM Architecture

Let's understand the internal architecture of JVM.
It contains : ① ~~class loader~~ classloader
② Memory area
③ Execution engine
④ Native Method Interface
⑤ ~~NM Interface~~

Date

Java Runtime System



→ Class Loader → loads class file
- → Bootstrap (rt.jar)
- → Extension (ext)
- → System Application (classpath).

→ Class Area → store pre-class structures.

→ PC register → stores address of currently executing instruction.

→ Execution Engine → Interpreter, compiler, all the functions of OS.

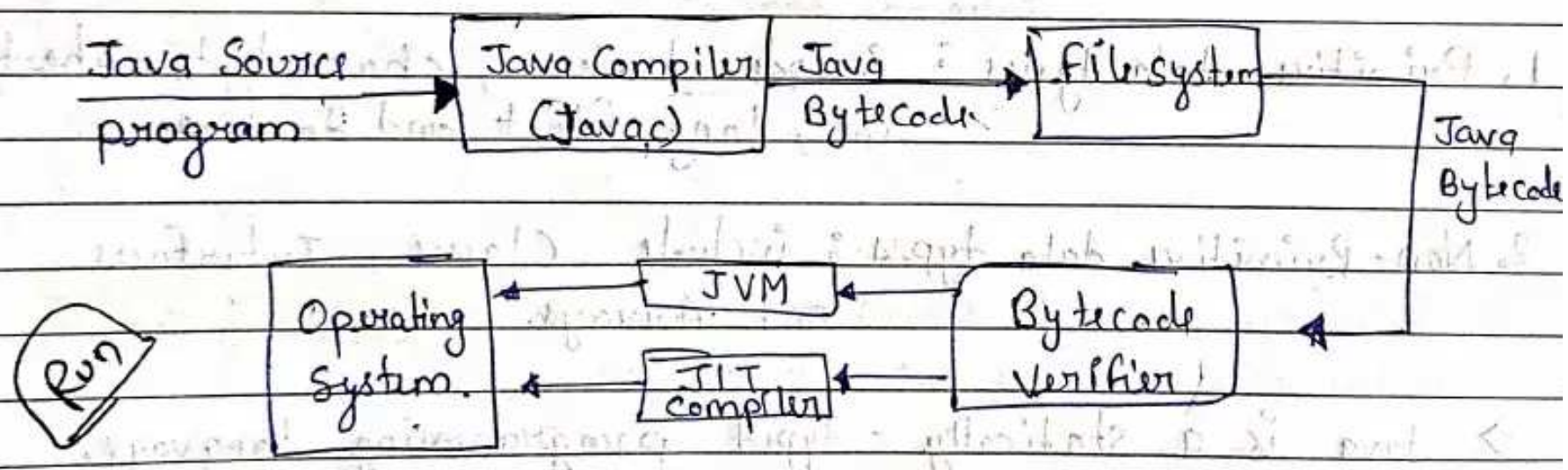
Date

Java Source file structure

→ It is used to describe that the Java Source code file must follow a structure.
→ The maximum no. of classes that may be declared as public in a Java program is one.
→ If a public class exists, the program's name and the name of the public class must match for there to be no compile time error.

Compilation and fundamentals

Write code in a text editor or IDE. Save as .java. Compile with ~ javac. Generates .class bytecode files. Execute bytecode with Java Command.



fundamentals → Class, object, Variables, Methods, Control flow statements, Packages.

Date

for writing a Java Program we have to follow some rules. The set of these rules is called Syntax.

Java Variables

A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

```
int data = 50;
```

Data Types in Java

→ Data types specify the different sizes and values that can be stored in the variable.

1. Primitive data types : include boolean, char, byte, short, int, long, float and doubles.

2. Non-Primitive data types : include Classes, Interfaces and Arrays.

→ Java is a statically - typed programming language. It means, all variables must be declared before its use and it's data type.

→ Java uses Unicode System not ASCII code System. The \u0000 is the lowest range of unicode system.

Date

Datatype	Default Value	Default Size
boolean	false	1 bit
Char	'\0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
Float	0.0F	4 byte
double	0.0d	8 byte

Java Comments.

→ Comments can be used to explain Java code. It can also be used to prevent execution when testing alternative code.

Single line → // ~~Comment~~ Comment.

Multi line → /* — — — —
— — — — */

Java Operators

→ operators are used to perform operations on variables and values.

→ Java divides operators into the following groups:

1. Arithmetic operators
2. Assignment operators
3. Comparison operators
4. Logical operators
5. Bitwise operators.

Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3

Comparison Operators

Comparison operators are used to compare two values (or variables). This is important in programming, because it helps us to find answers and make decisions.

The return value of a comparison is either true or false.

Operator	Name	Example
==	Equal to	x == y

!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Logical Operators

You can also test for true or false values with logical operators.

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

Logical Operators

You can also test for true or false values with logical operators.

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

Bitwise Operators

Bitwise operators are used to performing the manipulation of individual bits of a number. They can be used with any integral type (char, short, int, etc.).

Operation	Result	Same as	Result
5 & 1	1	0101 & 0001	0001
5 | 1	5	0101 | 0001	0101
~ 5	10	~0101	1010
5 << 1	10	0101 << 1	1010
5 ^ 1	4	0101 ^ 0001	0100
5 >> 1	2	0101 >> 1	0010

Control Flow in Java

- Java compiler executes the code from top to bottom.
- Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements.
- It is one of the fundamental features of Java, which provides a smooth flow of program.

Java provides three types of control flow statements.

1. Decision Making Statements -> if statements and switch statement
2. Loops Statements -> for, while, do while, for each loop
3. Jump Statements -> Break and Continue Statement

1) If Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false. In Java, there are four types of if-statements given below.

1. Simple if statement
2. if-else statement
3. if-else-if ladder
4. Nested if-statement

LETS CODE....

SWITCH:

In Java, a switch statement is a control flow statement that allows you to execute different blocks of code based on the value of a variable or expression.

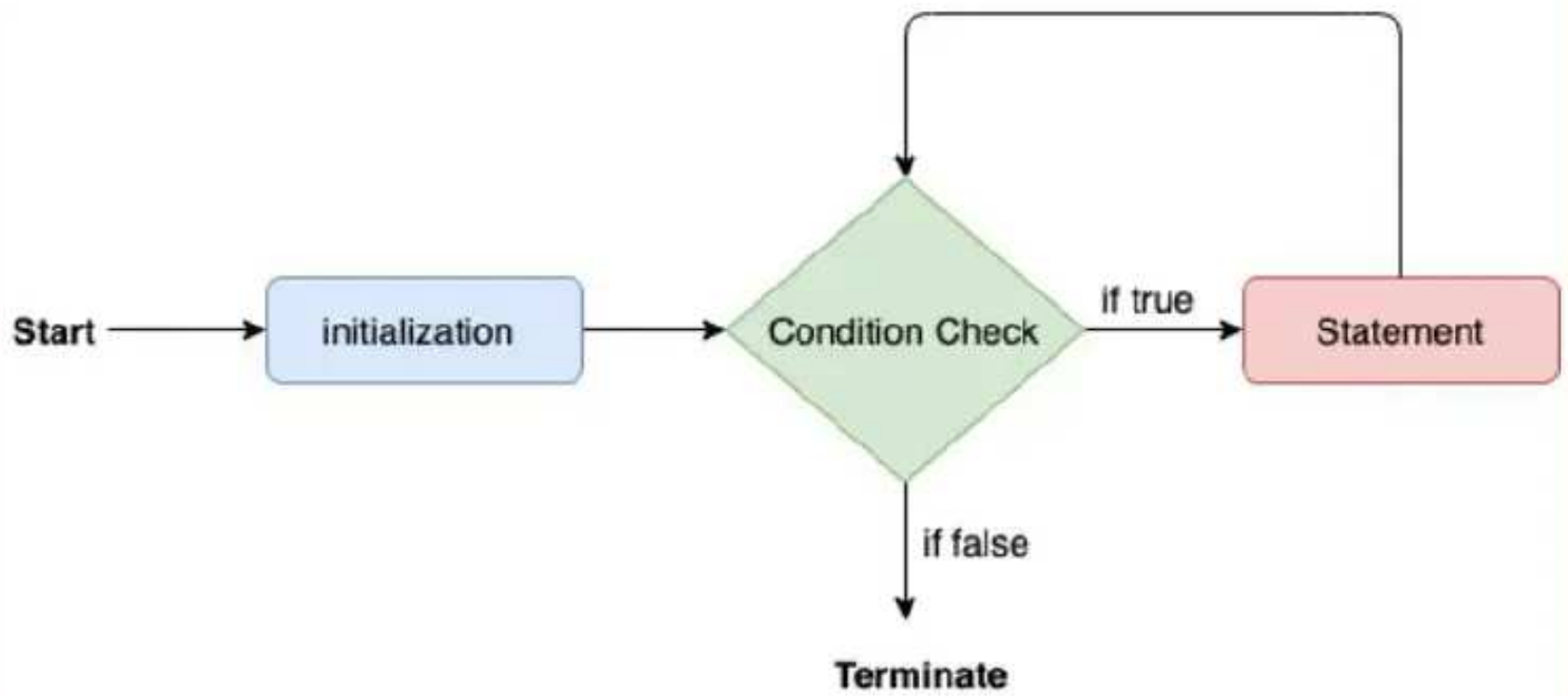
Loop Statements

In programming, sometimes we need to execute the block of code repeatedly while some condition evaluates to true. However, loop statements are used to execute the set of instructions in a repeated order. The execution of the set of instructions depends upon a particular condition.

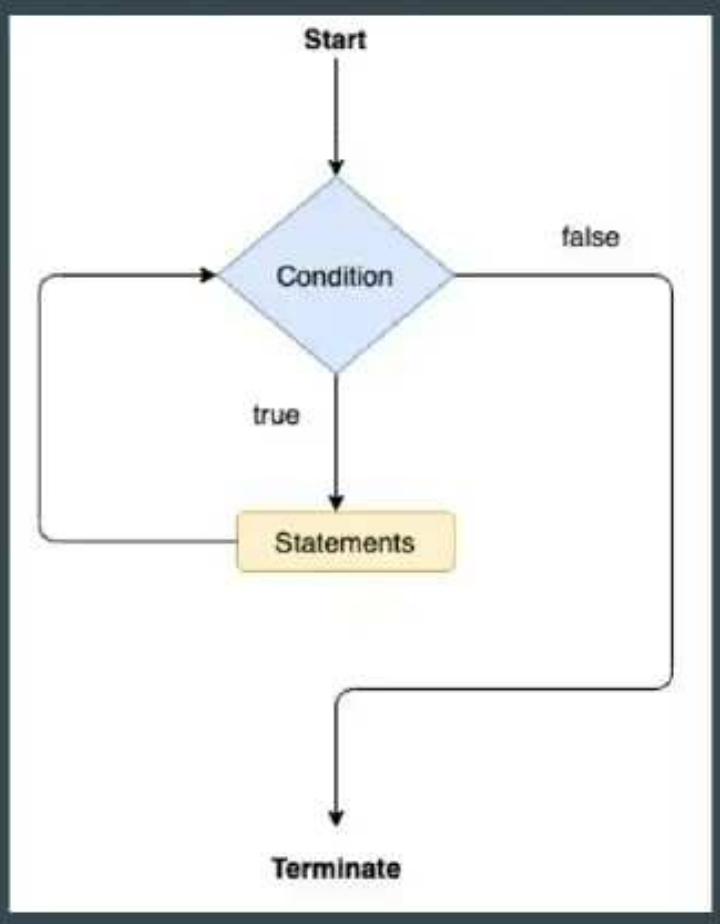
1. for loop
2. while loop
3. do-while loop
4. for-each loop

Lets Code...

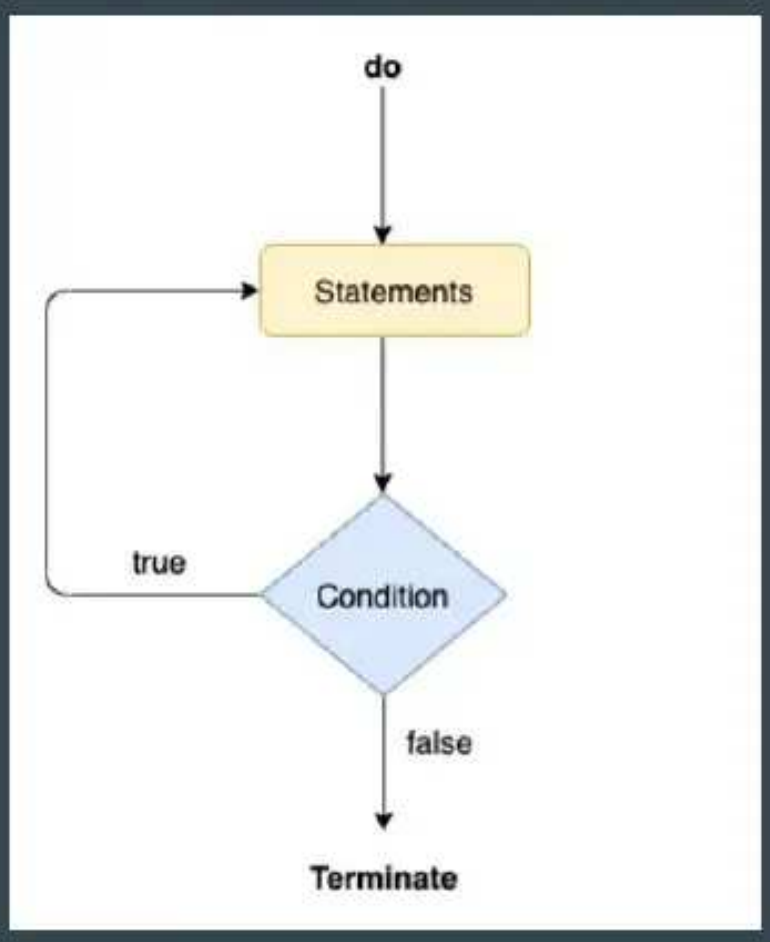
The flow chart for the for-loop is given below.



while loop flowchart



do-while loop flowchart



Jump Statements

Jump statements are used to transfer the control of the program to the specific statements.

Java break statement

the break statement is used to break the current flow of the program and transfer

the control to the next statement outside a loop or switch statement.

continue statement

Unlike break statement, the continue statement doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.

Arrays in Java

Java array is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.

Advantages:

Code Optimization: It makes the code optimized, we can retrieve or sort the data efficiently.

Random access: We can get any data located at an index position.

Disadvantages:

Size Limit: We can store only the fixed size of elements in the array. It doesn't grow its size at runtime.

String in Java

In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string.

How to create a string object?

There are two ways to create String object:

1. By string literal
2. By new keyword

OOPs (Object-Oriented Programming System)

Object means a real-world entity such as a pen, chair, table, computer, watch, etc. Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Class

A class is a basic building block. It can be defined as template that describes the data and behaviour associated with the class instantiation. Instantiating is a class is to create an object (variable) of that class that can be used to access the member variables and methods of the class.

An **object** in Java is the physical as well as a logical entity, whereas, a class in Java is a logical entity only.

Syntax:

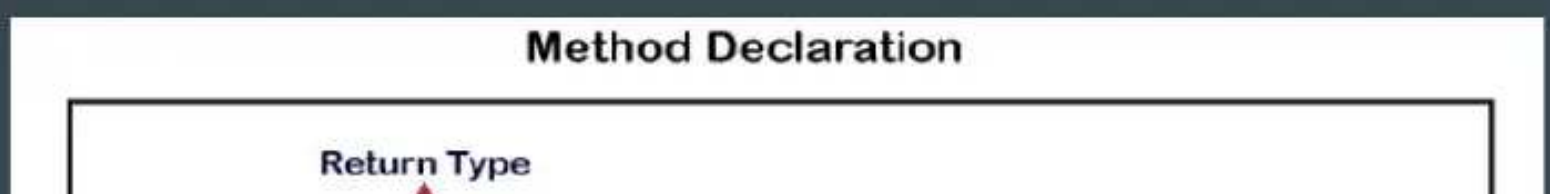
<access specifier> class class_name

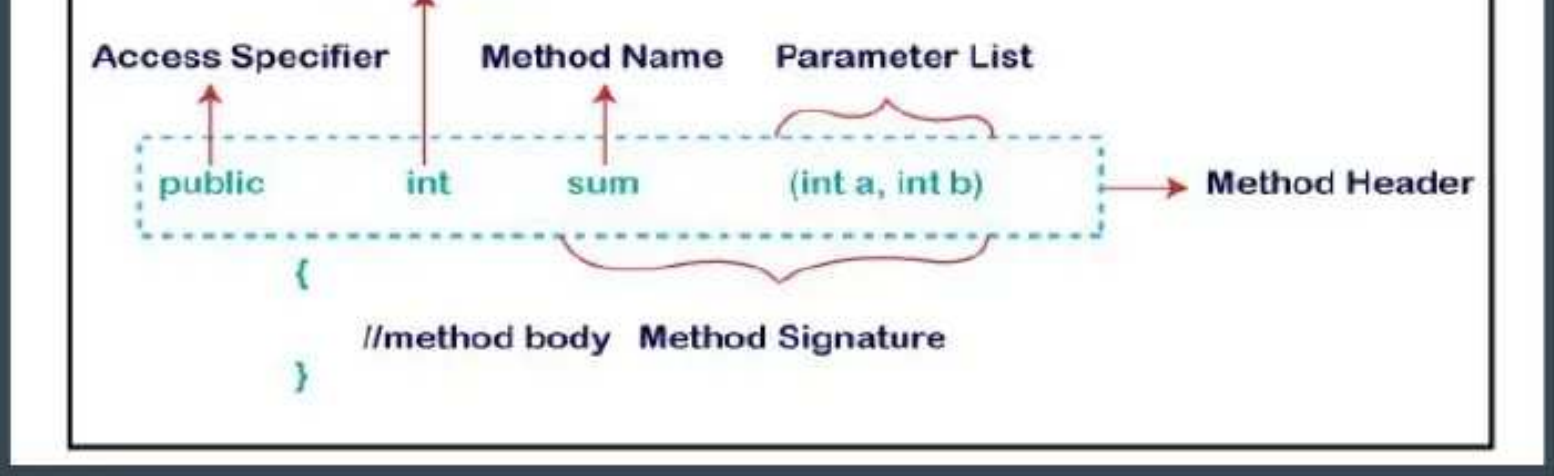
{

}

What is a method in Java?

A method is a reusable block of code designed to perform specific tasks. By calling or invoking it, we execute its functionality without needing to rewrite the code. This promotes code reusability, easy modification, and enhances code readability.





Access Specifier:

Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides four types of access specifier:

- **Public**: The method is accessible by all classes when we use public specifier in our application.
- **Private**: When we use a private access specifier, the method is accessible only in the classes in which it is defined.
- **Protected**: When we use protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **Default**: When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

Static Members:

- Static members belong to the class rather than to any specific instance of the class.
- They are shared among all instances of the class.
- Static members can be accessed directly using the class name without needing to instantiate an object.

Final Members:

- Final members are constants that cannot be changed once initialized.
- For variables, once a final variable is assigned a value, it cannot be reassigned.
- For methods, a final method cannot be overridden by subclasses.
- For classes, a final class cannot be subclassed.
- Final members are often used to define constants or to prevent further modification of certain elements in a program.

Constructors in Java

A constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

It is a special type of method which is used to initialize the object.

Rules for creating Java constructor

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. A Java constructor cannot be abstract, static, final, and synchronized

Types: Default and Parameterised

Inheritance in Java

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

The idea behind inheritance in Java is that you can create new classes that are built upon existing classes.

Why use inheritance in java

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Terms used in Inheritance

- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass

inherits the features. It is also called a base class or a parent class.

- **Reusability**: As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class.

The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
{
  //methods and fields
}
```

The **extends keyword** indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to **increase the functionality.**

Method Overloading

If a class has multiple methods having same name but different in parameters, it is known as Method Overloading.

There are two ways to overload the method in java:

- By changing number of arguments
- By changing the data type

Method Overriding

If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.

Rules for Java Method Overriding:

1. The method must have the same name as in the parent class
2. The method must have the same parameter as in the parent class.
3. There must be an inheritance.

Encapsulation

Encapsulation in Java is a process of wrapping code and data together into a single unit, for example, a capsule which is mixed of several medicines.
We can create a fully encapsulated class in Java by making all the data members of the class private. Now we can use setter and getter methods to set and get the data in it.

The Java Bean class is the example of a fully encapsulated class.

Polymorphism

Polymorphism in Java is a fundamental concept in object-oriented programming (OOP) that allows objects of different classes to be treated as objects of a common superclass. This allows for flexibility and extensibility in your code.

(Think of it this way: you have a group of superheroes with different abilities, but they can all be referred to as "superheroes".)

There are two main types of polymorphism in Java:

1. **Compile-time Polymorphism:** This is achieved through method overloading.
2. **Runtime Polymorphism:** This is achieved through method overriding.

Abstraction

Abstraction in Java is another core concept of object-oriented programming (OOP), alongside encapsulation, inheritance, and polymorphism. It involves hiding the complex implementation details of a system and showing only the essential features of the object.

Abstraction can be achieved in Java using two main mechanisms:

1. Abstract Classes
2. Interfaces

Abstraction allows you to focus on what an object does rather than how it does it, promoting code reusability, maintainability, and flexibility in large software systems.

Abstract Classes

An abstract class is a class that cannot be instantiated on its own and may contain abstract methods (methods without a body) as well as concrete methods (methods with a body). Abstract methods serve as placeholders for methods that must be implemented by subclasses. Abstract classes can have constructors and instance variables just like any other class.

```
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a circle");
    }}
```

In this example, Shape is an abstract class with an abstract method draw(). Subclasse Circle extend the Shape class and provide concrete implementations of the draw() method.

Interfaces

An interface in Java is a blueprint of a class that defines a set of abstract methods. Unlike abstract classes, interfaces cannot have instance variables or constructors. Classes can implement multiple interfaces, allowing them to inherit behavior from multiple sources.

```
interface Animal {
    void sound();
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }}
```